Code Debugging and Optimization

Project Report: C++ 2D Game Enemy Target Selector

Scope note: The task instructions say a code snippet will be provided, but none was included in the task text available. This report uses a self-contained illustrative example. If the provider later supplies the intended code, revise this report to analyze that code instead.

1. Objective

Analyze a game-related C++ function, identify correctness and efficiency issues, refactor it while preserving its intended behavior, and document the changes and tests.

2. Original behavior and issues

The sample function is intended to return the index of the closest active enemy to a player in 2D space, or -1 if no active enemy exists. The original initializes nearestDistance to 0.0f. Since Euclidean distances are non-negative, no candidate can be strictly less than zero; the function therefore returns -1 even when active enemies exist. This is the main correctness bug.

The original also calls std::sqrt for every active enemy even though square roots are unnecessary for comparing distances: squaring preserves the ordering of non-negative distances. It also uses an int loop counter when indexing a vector, whose size type is std::size_t.

3. Original code excerpt

```
#include <cmath>
#include <vector>
struct Vec2 { float x, y; };
struct Enemy { Vec2 position; bool active; };

int findNearestEnemy(const Vec2& player, const std::vector<Enemy>& enemies) {
    float nearestDistance = 0.0f; // BUG: distances cannot be less than zero
    int nearestIndex = -1;
    for (int i = 0; i < static_cast<int>(enemies.size()); ++i) {
        if (!enemies[i].active) continue;
        float dx = enemies[i].position.x - player.x;
        float dy = enemies[i].position.y - player.y;
        float distance = std::sqrt(dx * dx + dy * dy); // unnecessary sqrt
        if (distance < nearestDistance) {
            nearestDistance = distance;
            nearestIndex = i;
        }
    }
    return nearestIndex;
}
```

4. Refactored implementation

```
#pragma once
#include <cstddef>
#include <limits>
#include <vector>

struct Vec2 { float x{}; float y{}; };
struct Enemy { Vec2 position; bool active{true}; }

// Returns the closest active enemy index, or -1 if none exists.
inline int findNearestEnemy(const Vec2& player, const std::vector<Enemy>& enemies) {
    float bestDistanceSquared = std::numeric_limits<float>::infinity();
    int bestIndex = -1;

    for (std::size_t i = 0; i < enemies.size(); ++i) {
        const Enemy& enemy = enemies[i];
        if (!enemy.active) continue;

        const float dx = enemy.position.x - player.x;
        const float dy = enemy.position.y - player.y;
        const float distanceSquared = dx * dx + dy * dy;

        if (distanceSquared < bestDistanceSquared) {
            bestDistanceSquared = distanceSquared;
            bestIndex = static_cast<int>(i);
        }
```

```
        }
        return bestIndex;
    }
```

5. Annotated changes

Change	Reason / effect
Initialize best distance to infinity	Lets the first active enemy become the initial best candidate; fixes the correctness bug.
Compare squared distances	Avoids square-root calculations while preserving the nearest-enemy ordering.
Use std::size_t	Matches vector indexing/size types and avoids signed/unsigned warnings.
Use const Enemy&	Avoids copying each enemy during iteration.
Keep inactive filtering and -1 fallback	Preserves intended behavior for inactive enemies and empty input.

6. Testing and debugging process

The included tests cover the nearest active enemy, no active enemies, an empty list, and a single active enemy. The assertions are in tests/test_target_selector.cpp. Compile and run locally; only mark tests as passed after observing successful execution.

```
g++ -std=c++17 -Isrc tests/test_target_selector.cpp -o target_selector_tests\n./target_selector_tests
```

7. Performance analysis

The refactored function remains O(n) time and O(1) auxiliary space. It removes one square-root operation per active enemy and avoids copying each enemy. No before/after timing values are claimed because a benchmark has not been run here. For a valid benchmark, use an optimized build, a large repeatable dataset, multiple runs, and report the compiler, CPU, build flags, dataset size, and median runtime. For small input sizes, timing noise can exceed the measured difference.

8. Further improvements

Possible improvements include validating non-finite coordinates, adding an attack-range cutoff using squared distance, deterministic tests with fixed inputs, and profiling the actual game loop. For very large worlds, a spatial grid, quadtree, or engine-provided perception system may reduce the number of candidates scanned.

9. Conclusion

This example demonstrates a correctness fix and a small, justified optimization while retaining the intended nearest-active-enemy behavior. The package includes code, tests, build instructions, and this report. Because the task page did not include the promised snippet, the example is illustrative rather than a claim about the provider's missing code.

Before submission: run the tests in Visual Studio or another C++17 compiler, record the actual test result, and confirm that a self-selected example is acceptable.